

Part A - Architecture: BESS Operational AI-Agent

1. Agent Architecture

The agent operates on a continuous 15-minute rolling horizon. It maintains two parallel data pipelines:

- Real-Time Pipeline:** Ingests live data streams (State of Charge (SoC), current net load, instantaneous grid price) with sub-minute latency. This pipeline powers immediate real-time validation and reactive advisory responses.
- Forecast Pipeline:** At every new market gate (typically once per hour), the agent fetches updated 24-hour ahead forecasts for PV generation, four community loads (Office, Logistics, Manufacturing, and EV charging), and electricity prices. The agent uses a rolling window (e.g., the next 4 hours) for short-term operational decisions.

Interaction flow: When a user asks a query (e.g., advising on dispatch setpoints), the LLM parses the user intent into structured JSON arguments. It sends these parameters to deterministic tools (BESS Simulator, Cost Estimator) that query the cache containing the latest Pyomo optimizer run. The tools execute Python physics calculations and return structured values to the LLM, which translates these numeric logs into plain-language advice with clear mathematical provenance.

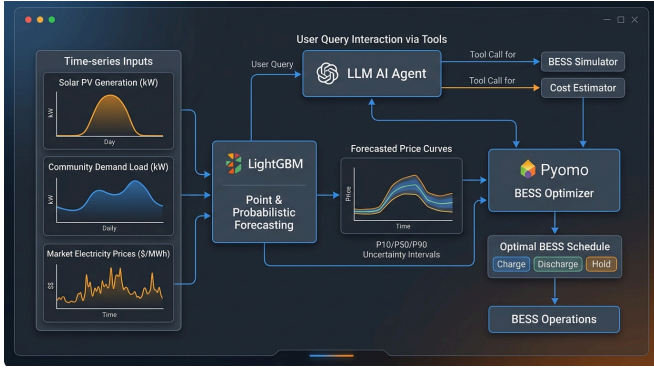


Figure 1: Operational architecture of the BESS AI-Agent.

Immediate vs. computationally intensive optimization: Both needs are met by decoupling the heavy pipeline (forecasting load, PV, and price, then re-solving the Pyomo optimizer) from the chat loop, so no query blocks on a solver run:

- Instant advisory (<1 s):** The agent reads the pre-computed optimal schedule directly from the shared cache to answer routine status and advisory queries immediately without invoking background processes.
- Asynchronous re-optimization (background task):** A strategic re-optimization request runs in the background rather than blocking the chat. The LLM answers right away using the last saved schedule, noting that it may be out of date, while a separate process recalculates the forecasts and reruns the Pyomo optimizer to produce an updated one.

2. Data Integration

The community comprises four load classes plus PV, grid/market, and BESS state, each with its own pattern and refresh cadence:

Source	Key pattern (15-min granularity)	Refresh
Office	08–18 weekdays, lunch dip 12–13	RT 5 min + hourly fc
Logistics	AM/PM activity waves + refrigeration base load	RT + schedule fc
Manufacturing	3-shift, start-up surges, seasonal ± 20 –40%	SCADA + shift API
EV (passenger)	evening/AM arrival peaks; 7/22 kW mix	Charger API + bookings
EV (HGV)	overnight depot 00–04h, 250 kW DC	Fleet dispatch API
PV	P10/P50/P90 24 h-ahead forecast	Weather API + SCADA
Grid / market	day-ahead price (e.g. SE3/SE4), Point of Common Coupling (PCC) import/export cap	Nord Pool / DSO
BESS	SoC, power limits, degradation state	BMS API, 5 s

RT: Real-Time, fc: Forecast

3. Decision Support Logic

Natural Language (NL) $\rightarrow$ recommendation. The LLM parses a query into structured intent (e.g. {horizon: 120min, objective: cost, ...}), calls the matching tool, and composes a response combining the tool's numbers with plain-language reasoning ("price rises after 17:00 and SoC is at 62%, so discharge now").

Rule-based vs. optimizer. A deterministic priority tree gives a safe, always-available fallback; the same priority order is encoded as hard constraints (safety, grid limit) ahead of the LP's soft cost objective, so the two layers never disagree on what is **allowed**, only on what is **optimal**:

Tier	Condition	Action
0 Safety	SoC $\leq 20\%$ / SoC $\geq 95\%$ & PV > Load	Force charge / discharge
1 Arbitrage	price $\ll / \gg$ forecast avg	Charge / discharge
2 Peak-shave	forecast peak > 85% of grid cap	Pre-charge BESS to prepare peak shaving
3 Solar harvest	surplus now, or SoC > 80% & future surplus	Charge / gentle discharge
4 Default	none of the above	Idle

Multi-objective balance. The agent balances three conflicting goals using a weighted sum objective $J = w_1 J_1 + w_2 J_2 + w_3 J_3$, where the weights w_i can be adjusted dynamically by the operator via chat:

- Cost Minimization (J_1 , weight w_1):** Minimizes the total cost of imported electricity net of degradation.
- Self-Consumption (J_2 , weight w_2):** Maximizes the fraction of PV generation consumed locally (avoiding cheap solar export).
- Grid Support / Peak Shaving (J_3 , weight w_3):** Minimizes peak grid demand.

The physical PCC grid limit is enforced as a **hard constraint** rather than a soft term, so peak-shaving overrides arbitrage whenever the limit is at risk. The agent explicitly communicates these trade-offs to the operator (e.g., "I prioritized cost minimization (50% weight) over self-consumption (30%) today due to high price volatility. However, I reserved 30% of battery capacity to ensure the 500 kW grid limit is never breached"). EV/HGV flexibility extends the LP as additional shiftable variables.

4. Uncertainty Handling

The agent manages forecast uncertainty through three primary mechanisms:

- Communicating forecast uncertainty to users:** The day-ahead spot price/load is forecast using a LightGBM model for the median (P50) point estimate, with P10/P90 quantiles constructed using **value-binned out-of-fold residuals bootstrapping**. The agent translates these quantile forecasts (P10, P50, P90) into clear natural language bounds: e.g., Recommendation: "Discharge 120 kW for the next 2 hours." Uncertainty Statement: "The PV forecast has a $\pm 15\%$ uncertainty (P10–P90 range) due to overcast conditions. If the sun underperforms, you may need to import an additional 30 kW from the grid to compensate."
- Robust decision-making under uncertain PV/load forecasts:** A conservative **Chance-Constrained Optimization** policy ensures the probability of violating the lower bound (SoC < 20%) is under 5% over the horizon. In practice, the agent raises the effective minimum SoC to 25–30% during high forecast variance, guaranteeing the battery is not catastrophically depleted.
- Learning from prediction errors over time:** The agent maintains a persistent error logger comparing forecasted values against realized actuals:
 - Bias Correction:** If the system consistently over-estimates PV generation by 5% on cloudy mornings, the agent applies a dynamic bias correction factor to future forecasts (automatically adjusted via an online moving-average filter).
 - Adaptive Weights:** The agent tracks which objective (cost vs. self-consumption) yielded better actual savings over the last 7 days and proposes weight shifts accordingly (e.g., "I noticed you exported 200 kWh unnecessarily yesterday. I suggest increasing the self-consumption weight from 0.3 to 0.4 for today").